
hammindist

Release 0.1.0

Students

May 25, 2026

CONTENTS:

1 Solvers API	1
----------------------	----------

SOLVERS API

`hammindist.functions.A(n, d, verbose=True)`

Calculate $A(n, d)$:param n : the length of the code :type n : int :param d : the minimum distance in the final set :type d : int

Returns

the length of the set (number of elements) code (set): the set with the elements at minimum distance d

Return type

size (int)

`class hammindist.functions.HammingTupla(lenght, distance)`

A class to generate binary tuples of a given length with Hamming weight less than or equal to a specified distance.

lenght(*int*)

The lenght of the binary tuples.

distance(*int*)

The maximum number of 1s (Hamming weight) allowed in each tuple.

graph(*networkx.Graph*)

The graph representation of the Hamming tuples, where nodes are tuples and edges connect tuples with Hamming distance 1.

as_list()

Return all generated tuples as a list.

Returns: list of tuple of int

A list containing all generated binary tuples.

draw()

Render the Hamming graph using Matplotlib without node labels.

Useful for larger graphs where text labels would clutter the visualization. Nodes are drawn as small ‘orchid’ colored circles.

get_instances_complete()

Generate all binary tuples of the given lenght (2^{lenght} tuples). This method is independent of the distance attribute.

Yields

tuple of int – A binary tuple (containing 0s and 1s).

get_instances_restricted()

static hamming_distance(*X*, *Y*)

Calculate the distance between two elements with the same length Arg:

X (List[any]): a string, an array or a tuple of 0's and 1's with length *n* *Y* (List[any]): a string, an array or a tuple of 0's and 1's with length *n*

Returns

a integer value non negative

Return type

d (int)

`hammindist.functions.accept(current_code, candidate, T)`

Simulated annealing acceptance criterion.

- Improvements are always accepted.
- Worse solutions may be accepted with probability $\exp(-T)$, where *T* is the size difference between solutions.

Parameters

- **current_code** (List[Any]) – Current solution (list of codewords). Typically the best-known or current working solution.
- **candidate** (List[Any]) – Candidate (neighbor) solution being evaluated. Usually generated by a perturbation or local move from the current solution.
- **T** (float) – Current temperature parameter in simulated annealing. Higher temperatures increase the probability of accepting worse solutions. As *T* decreases toward 0, the algorithm becomes more greedy (only accepts improvements).

Returns

Either the candidate solution (if accepted) or the

current_code (if rejected). Returns candidate for improvements or when probabilistic acceptance succeeds.

Return type

List[Any]

`hammindist.functions.binary_hamming_distance(x: int, y: int) → int`

`hammindist.functions.build_adjacency_bitsets(n: int, d: int, even_only: bool = False)`

Build an adjacency bitset representation of a graph where vertices are binary strings of length *n*, and edges connect vertices whose Hamming distance is at least *d*. :param *n*: Length of binary strings (number of bits). :type *n*: int :param *d*: Minimum Hamming distance required for an edge between vertices. :type *d*: int :param *even_only*: If True, only include vertices with even parity

(even number of 1-bits). Defaults to False.

Returns

Adjacency list as bitsets. adj[i] is an integer whose

j-th bit (from LSB) indicates if vertex *i* is connected to vertex *j*. Only bits for *j* > *i* are stored (lower triangular representation).

N (int): Number of vertices in the graph. *vertices* (List[int]): List of vertex values (integers) in the same order

as their indices in the adjacency list.

Return type

adj (List[int])

hammindist.functions.**build_outside_pool**(all_words_set, candidate, tabu_expiry, step)

Build a pool of candidate codewords for insertion.

Only codewords that: - Are not part of the current candidate solution. - Are not tabu.

The resulting pool is shuffled to encourage exploration of the search space.

Parameters

- **all_words_set** (Set[int]) – Set of all possible codewords in the search space (e.g., all binary strings of length n that satisfy constraints).
- **candidate** (List[int]) – Current candidate solution containing codewords already selected.
- **tabu_expiry** (Dict[int, int]) – Dictionary mapping codewords to the step number when their tabu status expires. A codeword is considered tabu if its expiry step > current step.
- **step** (int) – Current iteration step number. Used to determine which codewords have expired tabu status.

Returns

A shuffled list of codewords eligible for insertion.

Returns an empty list if no eligible codewords exist.

Return type

List[int]

hammindist.functions.**greedy_color**(P_bits, adj)

Greedy coloring of the subgraph induced by a set of vertices.

This function performs a greedy graph coloring on the subgraph containing only the vertices specified by P_bits. The coloring order is optimized by sorting vertices by their degree within the induced subgraph (highest degree first).

Parameters

- **P_bits** (int) – Bitset representing the set of vertices to include in the subgraph. Bit i is set if vertex i is present.
- **adj** (List[int]) – Adjacency bitsets for the full graph. adj[i] is an integer where bit j indicates an edge between vertices i and j (assuming j > i for upper triangular storage).

Returns

List of vertices sorted by color in

descending order (vertices with higher color numbers first).

num_colors (int): Number of colors used in the coloring.

Return type

sorted_verts (List[int])

hammindist.functions.**greedy_initial_clique**(adj, N, vertices)

Find a large clique using a greedy heuristic to obtain a lower bound. :param adj: Adjacency bitsets for the graph. adj[i] is an integer

where bit j indicates an edge between vertices i and j .

Parameters

- **N** – Number of vertices in the graph (length of adj list).
- **vertices** – List of vertex values (integers) corresponding to indices 0..N-1 in the adjacency list. This parameter is primarily for interface consistency with other functions, but not used directly in the heuristic.

Returns

Size of the found clique (number of vertices). `clique_bitset (int)`: Integer bitset where bit i is set if vertex i

(index in adjacency list) belongs to the clique.

Return type

size (int)

`hammindist.functions.greedy_start(all_words, d)`

Build an initial valid code using a greedy random approach.

Shuffles all candidate codewords and adds each one to the code if it satisfies the minimum distance constraint with all already-selected codewords.

This is used as a warm start for more sophisticated heuristics such as simulated annealing.

Args: `all_words` (List[int])

All candidate binary codewords of length n .

d (int):

Minimum Hamming distance required between any two codewords.

Returns: `code` (List[int]):

A valid code built greedily. Not guaranteed to be optimal.

`hammindist.functions.heuristic(n, d, iterations=1000, seed=None)`

Estimate $A(n, d)$ — the maximum size of a binary code of length n and minimum Hamming distance d — using a greedy heuristic with random restarts.

The algorithm builds a valid code greedily: at each step it tries to add a random codeword that satisfies the minimum distance constraint with all already-selected codewords. When no more codewords can be added, it records the size and restarts.

This is a heuristic, so the result is a lower bound on $A(n, d)$. It is not guaranteed to find the true optimum.

Args: `n` (int):

Length of the binary codewords.

d (int):

Minimum Hamming distance required between any two codewords.

iterations

[int, optional] Number of random restarts (default is 1000).

seed (int or None)

Random seed for reproducibility (default is None).

Returns:

best_code (List[int]):

The largest valid code found.

`hammindist.functions.is_valid_set(candidate_set, new_code, min_distance)`

Valid if a candidate_set is a valid set, it means that the elements of the set are at least the required distance apart
:param candidate_set: a set of codes with min_distance between them :type candidate_set: set :param new_code: a code that will be added to the set if it has min_distance with all

the elements of th candidate_set

Parameters

min_distance (int) – the minimum distance that must havee the all elements of the set with the new code

Returns

True if the set is valid, False if the set is not valid

Return type

bool

`hammindist.functions.max_clique_tomita(adj, N, vertices, use_translation=True)`

Tomita's algorithm for the length of maximum clique :param adj: Adjacency bitsets for the graph. adj[i] is an integer

where bit j indicates an edge between vertices i and j.

Parameters

- **N (int)** – number of nodes in the graph
- **vertices (List[any])** – nodes to work

Returns

maximun size finded best_clique (List[int]): the best clique that the fuction was able to build

Return type

max_size (int)

`hammindist.functions.max_set(codes_list, min_distance)`

Build the set C wich contains all the strings that are a distance d between them :param codes_list: the strings, arrays or tuples that we want to know if are in the “best_set” :type codes_list: List[any] :param min_distance: the distance that must have the elements of the “best_set” :type min_distance: int

Returns

the set of strings with min_distance between them

Return type

best_set (set)

`hammindist.functions.max_set_bounded(length, min_distance)`

Build the set C wich contains all the strings that are a distance d between them using a upper bound :param codes_list: the strings, arrays or tuples that we want to know if are in the “best_set” :type codes_list: List[any] :param min_distance: the distance that must have the elements of the “best_set” :type min_distance: int

Returns

the set of strings with min_distance between them

Return type

best_set (set)

`hammindist.functions.plotkin_bound(length, distance)`

Compute the Plotkin upper bound for $A(n, d)$.

Counts the total pairwise distance of a code in two ways:

- from below: each of the $C(M, 2)$ pairs contributes at least d , so $S \geq M(M-1)/2 * d$.
- from above: each of the n bit positions contributes at most $M^2/4$ differing pairs, so $S \leq n * M^2 / 4$.

Combining both inequalities and solving for M yields $A(n, d) \leq 2 * \text{floor}(d / (2d - n))$, but only when $2d > n$. When $2d \leq n$ the bound does not apply and infinity is returned.

Parameters

- **length** (*int*) – n , the length of the codes.
- **distance** (*int*) – d , the minimum Hamming distance required.

Returns

the Plotkin upper bound on $A(n, d)$,
or `float('inf')` if the bound does not apply.

Return type

`int | float`

`hammindist.functions.select_words_to_remove(current_code, tabu_expiry, step, k)`

Select k codewords to remove.

Priority is given to codewords that are not currently tabu. If there are not enough non-tabu codewords, the remaining selections are completed with tabu codewords. :param `current_code`: List of codewords currently in the code.

Each codeword is typically an integer representing a binary string.

Parameters

- **tabu_expiry** (*Dict[Any, int]*) – Dictionary mapping codewords to the step number when their tabu status expires. A codeword is considered tabu if its expiry step is greater than the current step.
- **step** (*int*) – Current iteration step number. Used to determine which codewords have expired tabu status.
- **k** (*int*) – Number of codewords to select for removal.

Returns

A list of k codewords selected for removal. The selection
prioritizes non-tabu codewords, but may include tabu codewords if necessary to reach k selections.

Return type

`List[Any]`

`hammindist.functions.simulated_annealing(n, d, iterations=5000, T_start=1.0, T_end=0.01, tabu_tenure=20, max_perturbation=3, seed=None)`

Estimate $A(n, d)$ using simulated annealing with tabu memory.

At each iteration: 1. Remove between 1 and `max_perturbation` codewords. 2. Mark removed codewords as tabu for a fixed tenure. 3. Attempt to add up to `max_perturbation + 1` new codewords. 4. Accept or reject the candidate solution according to

the simulated annealing criterion.

A reheating mechanism is also applied whenever no improvement is observed for an extended number of iterations.

`hammindist.functions.sphere_packing_bound(length, distance)`

Compute the sphere-packing (Hamming) upper bound for $A(n, d)$.

Treats each codeword as the center of a Hamming ball of radius $t = (d-1)/2$. Because balls around distinct codewords are disjoint, their total volume cannot exceed the full space 2^n , giving $A(n, d) \leq 2^n / \sum_{i=0}^t \binom{n}{i}$.

Parameters

- **length** (*int*) – n , the length of the codes.
- **distance** (*int*) – d , the minimum Hamming distance required.

Returns

the sphere-packing upper bound on $A(n, d)$.

Return type

int

`hammindist.functions.try_add_codewords(candidate, pool, d, k)`

Attempt to add up to $k + 1$ compatible codewords.

A codeword is added only if it preserves the required minimum distance from all codewords already in the solution. :param candidate: Current solution list of codewords already

selected. This list will be modified in-place.

Parameters

- **pool** (*List[int]*) – List of available codewords to consider adding. These are typically from a neighborhood or perturbation operation.
- **d** (*int*) – Required minimum Hamming distance. Any new codeword must have distance at least d from all codewords already in the candidate solution.
- **k** (*int*) – Maximum number of codewords to add beyond the first one. The function will add at most $k+1$ codewords total.

Returns

The modified candidate solution list (same list object

as the input, modified in-place) with newly added compatible codewords appended.

Return type

List[int]

`hammindist.functions.upper_bound(length, distance)`

Compute the tightest available upper bound for $A(n, d)$.

Takes the minimum of the sphere-packing (Hamming) bound and the Plotkin bound. When Plotkin does not apply ($2d \leq n$) its value is infinity, so the sphere-packing bound is returned automatically.

Parameters

- **length** (*int*) – n , the word length in bits.
- **distance** (*int*) – d , the minimum Hamming distance required.

Returns

the tightest upper bound on $A(n, d)$ from the two methods.

Return type
int

PYTHON MODULE INDEX

h

hammindist.functions, ??